



Game Scripting

Administrative Details

Unit Code & Name: GAD170 Game Scripting

Awards: Diploma of Game Development, Associate Degree of Games Development, Bachelor of Games Development (Games Design) or (Games Programming)

Award Level: AQF 5

Delivery mode: Face to Face, Blended

Course Committee: School of Information Technology and Creative Computing Committee

Duration: 13 Weeks

Prerequisites: Course Entry

Core or Elective: Core

Credit Points: 10

Course Outcomes: Course Overviews

Unit Overview

Game scripting is a common method of controlling the behaviour and sequencing of events and interactions within a game or scenario. Scripting can be thought of as a discrete language that uses principles of mathematics, combined with rules of syntax and patterns, to provide instructions for a computer to follow. In order for a game's design to move from an idea on paper to an experience that a player can interact with in a video game, we must be able to translate our design into a language that a computer can execute. We also want the instructions contained within this language to be consistent, accurate and effective, and we must possess an understanding of the mathematics that drive a computer, so that we can best utilise them.

In this module, you will implement a range of basic game functionality by utilising data manipulation, formulas, object creation, defining of behaviours, and integrating game engine features. You will also build an awareness of how to use these elements within common game scripting languages and environments.

Unit Learning Outcomes

LO Code	Learning Outcome
LO1	Plan program steps, identify stimuli of actions, describe effects of these actions and evaluate the success of your implementation.
LO2	Apply fundamental procedural programming techniques to manipulate game data and game loops.
LO3	Demonstrate the ability to script interactions with basic game engine features.
LO4	Describe, apply and evaluate basic Object Oriented programming concepts and skills in a game engine.

Student Workload

The learning outcomes in this unit can be achieved in approximately 120 hours over 13 weeks. Learning activities will normally include: face-to-face learning on campus or online, self-paced online learning, research and investigation, practising skills, completing assessment tasks, working with fellow students, self-directed study and transferable skills.

Schedule

Week	Topic	Project
0	Getting Started.	
1	Unity and Scripting Intro, Flow Charts, Project One.	
2	Operators, Random, If Statements, Inputs.	
3	Casting, Functions, UI: Buttons & Text.	
4	Intro to OOP, Getting References, Null Checks, Turning Objects On/Off And Audio	
5	Materials, Animations, Particles, Audio, Post Project Reflection.	
6	Prefabs, Instantiation and Destruction, Project Two	Project 1 Due
7	Study Week.	
8	Rigidbody, Triggers, Collisions, Project Two	
9	Loops, Arrays and Lists.	
10	Coroutines, Scene Loading, Exporting Your Build.	
11	Custom Classes, Getters and Setters, Switches, Enums.	
12	Unity Events and Delegates.	Project 2 Due
13	What's Next.	

Holistic Assessment

At SAE, assessments encourage students to investigate solutions to real-world problems and construct knowledge from inquiry-based processes and collaborative activities. Your facilitator will assess your work holistically throughout the unit.

1. **To achieve a pass:**

- **All projects must be submitted to be eligible for a Pass grade in the unit overall.**
- **All learning outcome requirements in each project brief must be met.**

2. **Grades above a pass:** Once the pass threshold for a project has been met, the standard of the project requirements will be holistically assessed (Pass, Credit, Distinction and High Distinction).

3. **Unit Grade:** Your final grade will be the average of your project grades.

Late Submissions & Special Consideration

If you have questions or difficulties with your assessments, speak to your facilitator as soon as possible. To request an extension or alternative assessment, use the [Special Consideration form](#).

Learn more about assessment at SAE in your [Student Handbook](#), including info on:

- Project grades
- Late Submissions & Special Consideration
- Assessment Re-mark
- Academic Integrity
- Group Work Agreement

Project Overview

Project	Starts	Deadline	Learning Outcomes	Transferable Skills	Graduate Attributes
Project 1	Week 1	Week 6	LO1, LO2, LO3, LO4	1, 2, 3, 4, 5, 7 & 8	1-6
Project 2	Week 6	Week 12	LO1, LO2, LO3, LO4	1, 2, 3, 4, 5, 7 & 8	1-6

Project 1 Brief

Game Data and Flow

Due	Week 6 - Sunday 11:59pm
Group / Individual Project	Individual
Artificial Intelligence Use:	Level 1 - No AI

Brief:

To a large extent, games are driven by numbers and the manipulation of these numbers. Making use of variables, functions and control flow is core to these manipulations, and is essential for building game functionality. Both analogue and video game designs and development make use of the manipulation of numbers. Developers continuously tweak these numbers to adjust the feel of their games to make them align with the design intent.

For example, the accumulation of XP (eXperience Points) in order to level up and allocate points to stats and skills, is entirely about number manipulation and balance.

Using these numbers to your advantage may already be familiar to you as a player, but we need to learn how we design and engineer these numbers as developers because many, many things happen behind the curtain, where players cannot see.

Project Goals:

You must create a C# Unity project that demonstrates the fundamental concepts of programming this includes demonstration and understanding of the following:

- Flowchart logic.
- Pseudo Code.
- Game Engine functionality such as attaching scripts to objects, using the console to debug messages, responding to input and outputting to UI.
- Code naming conventions such as Pascal Casing for script names, camelCasing for Variables.
- Commenting your code.
- Debugging your code.
- Creating variables and assigning values.
- Assigning and using random values.
- If statements and conditional logic.
- Using operators such as (`()`, `+`, `-`, `/`, `*`, `=`, `==`).
- Data conversions such as a float to an int.
- Creating and Utilising Functions
- Changing the value of a variable or calling a function from another script.
- A basic win/loss condition that ends/restarts the game (i.e. game loop).
- Evaluating your processes and the implementation of your project.

Steps to Follow:

1. Open: GAD170 on Campus Online and click on the Project One Tab.
2. Follow: the step-by-step process outlined in the project outline document - It contains detailed steps on how to complete your project, what to submit, and how to submit.

Deliverables:

- Project Documentation as a PDF which includes:
 - Flow chart for your project.
 - Pseudo code for your project.

- Inclusion of your APA7 Reference.
- An MP4 video covering and discussing and reflecting on the following:
 - Showing the Authoritative source that you have reviewed and discuss what you learned from it and how you could apply it to your current project.
 - Showing your APA 7 Reference on screen.
 - Critically analysing your work and making suggestions for actionable steps for improvement:
 - Pseudo Code.
 - Flow Chart
 - Code and Unity Project.
 - Review your code and show where you have completed each of the program requirements in your submission.
 - Show the program running in Unity Editor, even if errors occur.
 - An estimation of the grade you believe you should receive based on your submission.
 - 3-7 mins max Length.
- A ZIP file of your Unity Project Files, Documentation PDF and Reflective Video.

How your Project will be graded:

Learning Outcomes	Deliverable
LO1 Plan program steps, identify stimuli of actions, describe effects of these actions and evaluate	• Flow Chart that: ◦ Demonstrates step wise logic and branching behaviours with actions. • Pseudo Code that: ◦ Demonstrates planning and provides rough guidance on the overall steps that your program will perform. • A MP4 Post Project Reflection Video, discussing and reflecting on the following: ◦ Pseudo Code.

the success of your implementation.	◦ Flow Chart ◦ Includes actionable steps that you could take to improve your project deliverables and work flows. ◦ Includes minimum one APA 7 Authoritative reference. ◦ 3-7mins Max.
LO2. Apply fundamental procedural programming techniques to manipulate game data and game loops.	• C# scripts that demonstrate each of the following at least once: ◦ Code naming conventions such as Pascal Casing for script names, camelCasing for Variables. ◦ Commenting your code. ◦ Debugging your code. ◦ Creating variables and assigning values. ◦ Assigning and using random values. ◦ Using operators such as (), + , - , / , * , = , ==). ◦ A basic win/loss condition that ends/restarts the game (i.e. game loop). ◦ The use of functions (any type) within your code.
LO3. Demonstrate the ability to script interactions with basic game engine features.	• A Unity Project that has: ◦ Scripts attached to objects. ◦ Using the console to debug messages. ◦ Responding to Input. ◦ Updating and displaying information to the UI.
	• A Unity project that demonstrates these at least once:

LO4 Describe, apply and evaluate basic Object Oriented programming concepts and skills in a game engine.	◦ Changing the value of a variable or calling a function from another script. • A MP4 video covering and discussing and reflecting on the following: ◦ Your project code. ◦ Includes actionable steps that you could take to improve your project deliverables and workflows. ◦ Includes minimum one APA 7 Authoritative reference. ◦ 3-7mins Max.
--	---

Project 2 Brief

Game States and Interactions

Due	Week 12 - Sunday 11:59pm
Group / Individual Project	Individual
Artificial Intelligence Use:	Level 1 - No AI

Brief:

Games are more than just pixels on a screen, they are made up of distinct objects with recurring properties. A door, for example, is not just something we see, it is something that can move, that can obstruct movement, that players and AI characters can interact with, that can emit sounds, and it shares these properties with other doors in the game. Everything that you see in a game - the buttons that you click, the individual trees in an environment, the player and AI characters - can be broken down into objects, and groups of objects. This is the case in the smallest indie titles

to the very largest AAA projects.

The objects we see on screen usually have scripts attached to them that hold information (e.g. their location, what state they are in, a character's run speed etc) as well as the functions to manipulate that information (e.g. to make the character move). There also are other objects present that we can't see, such as game 'managers', which keep track of the state of the world, may control background music, or may coordinate the actions of the AI characters.

For this project, you will be making use of Object-Oriented Programming (OOP) techniques by creating and working with various objects, using collections (e.g. arrays and lists) to store game objects, and using loops to access game objects and control game behaviour.

Project Goals:

You must create a C# Unity project that demonstrates the fundamental concepts of programming.

This includes demonstration and development of the following skills and concepts you learned in Project One and Two:

- Flowchart logic.
- Pseudo Code.
- Game Engine functionality such as attaching scripts to objects, using the console to debug messages, responding to input and outputting to UI.
- Code naming conventions such as camelCasing for Variables, Pascal Casing for script names and functions.
- Commenting your code.
- Debugging your code.
- Creating variables and assigning values.
- Assigning and using random values.
- If statements and conditional logic.
- Using operators such as (), + , - , / , * , = , ==).
- The use of functions (any type) within your code.
- Instantiation of GameObjects.
- Evaluating your processes and implementation of your project.
- Object Oriented Programming (OOP) concepts such as:

- Changing the value of a variable or calling a function from another script.
 - Checking for NULL before calling functions/accessing variables.
- Detection of stimuli using:
 - Triggers (Enter, Stay, Exit).
 - Collisions (Enter, Stay, Exit).
- Triggering of feedback systems such as:
 - Animations
 - Particles
 - Audio
 - Changing materials
 - Changing colliders
 - Changing physics

This project also requires some more advanced coding concepts including:

- The use of loops (any type) within your code.
- The use of collection i.e. array, list, dictionary etc.

Steps to Follow:

1. Open: GAD170 on Campus Online and click on the Project Two Tab.
2. Follow: the step by step process outlined in the project outline document - It contains detailed steps on how to complete your project, what to submit and how to submit.

Deliverables:

- Project Documentation as a PDF which includes:
 - Flow chart for your project.
 - Pseudo code for your project.
 - Inclusion of your APA7 Reference.
- An MP4 video covering and discussing and reflecting on the following:
 - Showing the Authoritative source that you have reviewed and discuss what you learned from it and how you could apply it to your current project.
 - Showing your APA 7 Reference on screen.

- Critically analysing your work and making suggestions for actionable steps for improvement::
 - Pseudo Code.
 - Flow Chart
 - Code and Unity Project.
- Review your code and show where you have completed each of the program requirements in your submission.
- Show the program running in Unity Editor, even if errors occur.
- An estimation of the grade you believe you should receive based on your submission.
- 3-7 mins max Length.
- A ZIP file of your Unity Project Files, Documentation PDF and Reflective Video.

How your Project will be graded:

Learning Outcomes	Deliverable
LO1 Plan program steps, identify stimuli of actions, describe effects of these actions and evaluate the success of your implementation.	• Flow Chart that: ◦ Demonstrates step wise logic and branching behaviours with actions. • Pseudo Code that: ◦ Demonstrates planning and provides rough guidance on the overall steps that your program will perform. • A MP4 Post Project Reflection Video, discussing and reflecting on the following: ◦ Pseudo Code. ◦ Flow Chart ◦ Includes actionable steps that you could take to improve your project deliverables and work flows.

	- Includes minimum one APA 7 Authoritative reference. 3-7mins Max.
LO2. Apply fundamental procedural programming techniques to manipulate game data and game loops.	- C# scripts that demonstrate each of the following at least once: - Code naming conventions such as Pascal Casing for script names, camelCasing for Variables. - Commenting your code. - Debugging your code. - Creating variables and assigning values. - Using operators such as ((), + , - , / , * , = , ==). - The use of functions (any type) within your code. - Instantiation of GameObjects within your code. - The use of loops (any type) within your code. - The use of a collection i.e. array, list, dictionary etc.
LO3. Demonstrate the ability to script interactions with basic game engine features.	- A Unity project that demonstrates these at least once: - Scripts attached to objects. - Using the console to debug messages. - Responding to Input. - Updating and displaying information to UI. - Triggering feedback systems through code, such as: - Animations - Particle Effects.

	■ Audio ■ Changing materials ■ Changing colliders ■ Changing physics
LO4 Describe, apply and evaluate basic Object Oriented programming concepts and skills in a game engine.	• A Unity project that demonstrates these at least once: ◦ Changing the value of a variable or calling a function from another script. ◦ Checking for NULL before calling functions/accessing variables. ◦ Triggers (either: Enter, Stay, Exit). ◦ Collisions (either: Enter, Stay, Exit). • A MP4 Post Project Reflection Video, discussing and reflecting on the following: ◦ Your project code. ◦ Includes actionable steps that you could take to improve your project deliverables and workflows. ◦ Includes minimum one APA 7 Authoritative reference. ◦ 3-7mins Max.

Required Readings

- Unity Technologies. (n.d.). Unity user manual 2022.3 (LTS). Unity User Manual 2022.3.
<https://docs.unity3d.com/Manual/index.html>
- Unity Technologies. (n.d.). Beginner scripting. Unity Learn.
<https://learn.unity.com/project/beginner-gameplay-scripting>
- Microsoft Corporation. (n.d.). C# docs - get started, tutorials, reference, Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/csharp/>

Recommended Readings

- Backhouse, R. (2011). Algorithmic problem solving. Wiley.
- Code Monkey. (n.d.). Unity Basics for Beginners [Video]. YouTube. <https://www.youtube.com/playlist?list=PLzDRvYVwl53vxdAPq8OznBAj0eeiipT>
- Roam, D. (2009). The back of the napkin: Solving problems and selling ideas with pictures. Portfolio.
- van Verth, J. M., & Bishop, L. M. (2016). Essential mathematics for games and interactive applications (3rd ed.). CRC Press.
<https://ebookcentral.proquest.com/lib/sae/detail.action?docID=4742705>

Specialised Resource Requirements

- Adobe Creative Suite or equivalent (Photoshop, After Effects, Premiere)
- Office Suite (Microsoft Office, GSuite)
- Microsoft Windows based computer with Computer Graphics capabilities (Powerful CPU, Graphics Card, sufficient RAM, Hard Drive space)
- Audio Video Capture Incl. Webcam, Microphone and Game recording software
- Game Engines (Incl. Unity and Unreal)
- IDE (Inc. Microsoft Visual Studio)

AI Assessment Scale

Level & Category	Description	Application in Creative Industries	Activities
Level 1 - No AI	Students do not use AI tools in their creative process.	Suitable for projects requiring personal expression, original craftsmanship, or the demonstration of foundational creative skills.	Activities: sketching, hand-drawing, analog photography, live performance, in-person critiques.
Level 2- AI-Assisted Idea Generation and Structuring	AI is used for brainstorming and organising ideas, but final creative work must be original and	Useful for concept development, mood boards, and initial drafts in design, writing, or multimedia projects,	Activities: collaborative brainstorming, creating project outlines, mood boards, AI-assisted research for inspiration,

	human-authored.	developing effective feedback language.	framing feedback for peers constructively.
Level 3- AI-Assisted Editing	Students use AI for refining, editing, or enhancing their creative outputs.	Beneficial for polishing content such as graphic design, music production, or film editing. AI assists with technical adjustments, but the creative vision remains human-directed.	Activities: correcting grammar/spelling in scripts, adjusting image filters, fine-tuning sound mixing, visual editing.
Level 4- AI-Enhanced Creative Task Completion	AI tools are integrated into specific aspects of creative projects, with students critically evaluating AI outputs and integrating them into their work.	Encourages exploration of AI as a collaborative partner in tasks like generating visual effects, automating repetitive tasks, or experimenting with generative design.	Activities: AI-generated visuals or music, comparative analysis of AI vs. human work, critical evaluation of AI-generated content, blending AI outputs with human creativity.
Level 5- Full AI Integration	AI is fully integrated throughout the creative process, allowing for deep exploration and	Suitable for experimental projects where AI plays a central role, pushing boundaries of creativity and exploring new mediums or	Activities: co-creating with AI (e.g., generative art, AI-driven music composition), real-time collaboration with AI tools, iterative feedback loops,

	innovation.	methods.	AI as a creative tool in producing final projects.
--	-------------	----------	--

Perkins, M., Furze, L., Roe, J., & MacVaugh, J. (2024). The artificial intelligence assessment scale (AIAS): A framework for ethical integration of generative AI in educational assessment. *Journal of University Teaching and Learning Practice*, 21(06).
<https://doi.org/10.53761/q3azde36>

Transferable Skill Readings

- Golden, D. L. (2016). Visual management of time. In A. Benedek & Á. Veszelszki (Eds.), *In the beginning was the image: The omnipresence of pictures: Time, truth, tradition* (pp. 51-57). Peter Lang.
<https://www.jstor.org/stable/j.ctv2t4cns.7>
- Pfeiffer, S., Sauer, S., & Ritter, T. (2019). Agile methods as stress management tools? an empirical study. *Work Organisation, Labour & Globalisation*, 13(2), 20-36.
<https://doi.org/10.13169/workorglaboglob.13.2.0020>
- Wankat, P. C., & Oreovicz, F. S. (2015). Problem solving and creativity. In *Teaching Engineering* (2nd ed., pp. 93–116). Purdue University Press.
<https://doi.org/10.2307/j.ctv15wxqn9.9> ().